SHOPSPHERE

AWS EC2 Microservices Deployment Handbook

A practical record of EC2 provisioning, SSH access, Docker installation, IAM and ECR, Docker Compose, Nginx, application verification, and AWS Systems Manager.

Item	Configuration
AWS Region	ap-southeast-2 (Sydney)
EC2	Ubuntu Server 24.04 LTS, t3.small, 8 GiB gp3
Instance	ShopSphere-Production-Server
Registry	Amazon ECR
Runtime	Docker Engine and Docker Compose
Public entry point	Nginx on TCP port 80
Remote management	SSM Session Manager and Run Command

Prepared from the deployment conversation. Where exact historical file contents were not retained, examples are identified as reconstructed reference configurations.

Contents

1. Architecture
2. EC2 provisioning and security group
3. Windows SSH key permissions
4. SSH connection
5. Ubuntu update and Docker installation
6. Docker group permissions
7. EC2 IAM role and ECR permissions
8. AWS CLI and ECR login
9. Pull images
10. Deployment directory
11. Docker Compose configuration
12. Environment variables and secrets
13. Nginx reverse proxy
14. Validate and deploy
15. Application verification
16. Systems Manager installation and IAM
17. Session Manager and ssm-user Docker access
18. SSM Run Command
19. Consolidated commands
20. Checklist
21. Production hardening
22. Interview questions

1. Architecture

ShopSphere runs on one Ubuntu EC2 instance. Jenkins previously built, tested and pushed four versioned application images to ECR. EC2 pulls those images and Docker Compose runs five containers. Nginx is the only public application entry point.

Service	Responsibility	Port
frontend	React/Vite UI	80 internal
user-service	Registration/login API	3001 internal
product-service	Product catalogue API	3002 internal
order-service	Order processing API	3003 internal
nginx	Public reverse proxy	80 public

Traffic flow: Browser → EC2 TCP/80 → Nginx → frontend or backend container. Backend ports are not published directly to the internet. Containers communicate through a Docker bridge network using Compose service names.

This is a single-instance learning deployment, not a highly available or fully production-hardened architecture.

2. EC2 provisioning and security group

Setting	Value
Name	ShopSphere-Production-Server
AMI	Ubuntu Server 24.04 LTS, 64-bit x86
Instance type	t3.small
Region	ap-southeast-2 (Sydney)
Key pair	shopsphere-key.pem
Storage	8 GiB gp3
Public IPv4	Enabled
Inbound SSH	TCP 22 from My IP
Inbound HTTP	TCP 80 from 0.0.0.0/0

HTTPS/443 was not configured at this stage. Observed public IP was 13.211.127.214; use the current EC2 console address because public IPs can change after stop/start unless an Elastic IP is used.

SSH was restricted to My IP. HTTP was open publicly to serve the website. Never expose backend ports 3001–3003 directly.

3. Windows SSH private-key permissions

Run in Windows PowerShell. Adjust the PEM path if needed.

```
$key = "$HOME\Downloads\shopsphere-key.pem"
$me = (whoami).Trim()
icacls $key /inheritance:r
icacls $key /grant:r "$($me):(R)"
icacls $key
```

icacls manages Windows ACLs. /inheritance:r removes inherited permissions; /grant:r replaces explicit permissions for the current user; (R) grants read access. SSH may reject a private key that is readable by other users.

Never share the PEM file, upload it to chat, or commit it to Git.

4. Connect to EC2 using SSH

```
ssh -i "$HOME\Downloads\shopsphere-key.pem" ubuntu@13.211.127.214
```

ssh starts the client; -i selects the private key; ubuntu is the default Ubuntu AMI user; the final argument is the public IP. Successful prompt looked like ubuntu@ip-172-31-37-171:~$.

5. Update Ubuntu and install Docker

Docker packages applications and dependencies into images and runs them as containers. Docker Compose coordinates the five services. The exact historical installation command sequence was not retained; the following is a reproducible installation sequence using Docker's official Ubuntu repository.

```
sudo apt update
sudo apt upgrade -y
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo systemctl start docker
sudo systemctl enable docker
sudo systemctl status docker
docker --version
docker compose version
sudo docker run hello-world
```

The hello-world container verifies the daemon can pull and run an image. Compose v2 uses docker compose (space), not the legacy docker-compose command.

6. Docker group permissions

```
sudo usermod -aG docker ubuntu
newgrp docker
docker ps
```

This adds ubuntu to the docker group so sudo is not required for Docker commands. Docker group membership effectively grants root-equivalent control of the host; only trusted administrators should be members.

7. EC2 IAM role for ECR

EC2 uses an IAM instance profile/role to authenticate to ECR without storing long-lived AWS access keys on the server. The role observed in the deployment was ShopSphere-ECR-Pull-Policy.

Permission	Purpose
ecr:GetAuthorizationToken	Obtain ECR authentication token (Resource *)
ecr:BatchCheckLayerAvailability	Check image layers
ecr:GetDownloadUrlForLayer	Download image layers
ecr:BatchGetImage	Retrieve image manifests

The custom pull policy was scoped to the four ShopSphere repositories. ecr:DescribeRepositories is not required for image pulls; an AccessDenied response to that separate action does not prove pulling is broken.

```
aws sts get-caller-identity
```

The output showed account 507941515612 and an assumed role similar to arn:aws:sts::507941515612:assumed-role/ShopSphere-ECR-Pull-Policy/i-09225500db5e74575.

8. AWS CLI and ECR authentication

```
aws --version
aws sts get-caller-identity
aws configure get region
```

```
aws ecr get-login-password --region ap-southeast-2 | \
docker login --username AWS --password-stdin \
507941515612.dkr.ecr.ap-southeast-2.amazonaws.com
```

Expected login output: Login Succeeded. The EC2 role provides AWS credentials automatically. ECR authorization tokens are temporary (normally 12 hours). Docker may store credentials in ~/.docker/config.json; do not share that file.

9. Pull the four images

```
docker pull 507941515612.dkr.ecr.ap-southeast-2.amazonaws.com/shopsphere-user:v1.0.0
docker pull 507941515612.dkr.ecr.ap-southeast-2.amazonaws.com/shopsphere-product:v1.0.0
docker pull 507941515612.dkr.ecr.ap-southeast-2.amazonaws.com/shopsphere-order:v1.0.0
docker pull 507941515612.dkr.ecr.ap-southeast-2.amazonaws.com/shopsphere-frontend:v1.0.0
docker images
```

Repository	Tag
shopsphere-user	v1.0.0
shopsphere-product	v1.0.0
shopsphere-order	v1.0.0
shopsphere-frontend	v1.0.0

Jenkins built/tested/pushed the images. EC2 only pulls and runs them; it does not build the application.

10. Deployment directory

```
mkdir -p ~/shopsphere
cd ~/shopsphere
mkdir -p nginx
pwd
```

```
shopsphere/
■■■ docker-compose.yml
■■■ .env
■■■ nginx/
    ■■■ nginx.conf
```

This keeps runtime deployment configuration separate from the application source repository.

11. Production Docker Compose

The following is a reconstructed reference configuration based on the deployment setup and observed output. It is not a byte-for-byte export of the live file; validate against the actual EC2 file before reusing.

```
services:
  user-service:
    image: 507941515612.dkr.ecr.ap-southeast-2.amazonaws.com/shopsphere-user:v1.0.0
    container_name: user-service
    restart: unless-stopped
    env_file: [.env]
    environment:
      PORT: "3001"
      DATA_DIR: /app/data
    volumes:
      - user-data:/app/data
    networks: [app-network]
    healthcheck:
      test: ["CMD", "node", "-e", "require('http').get('http://localhost:3001/health',r=>process.exit(r.statusCode===200?0:1)).on('er
      interval: 30s
      timeout: 10s
      retries: 3

  product-service:
    image: 507941515612.dkr.ecr.ap-southeast-2.amazonaws.com/shopsphere-product:v1.0.0
    container_name: product-service
    restart: unless-stopped
    environment:
      PORT: "3002"
    networks: [app-network]
    healthcheck:
      test: ["CMD", "node", "-e", "require('http').get('http://localhost:3002/health',r=>process.exit(r.statusCode===200?0:1)).on('er
      interval: 30s
      timeout: 10s
      retries: 3
```

```
  order-service:
    image: 507941515612.dkr.ecr.ap-southeast-2.amazonaws.com/shopsphere-order:v1.0.0
    container_name: order-service
    restart: unless-stopped
    env_file: [.env]
    environment:
      PORT: "3003"
      DATA_DIR: /app/data
      PRODUCT_SERVICE_URL: http://product-service:3002
    volumes:
      - order-data:/app/data
    networks: [app-network]
    healthcheck:
      test: ["CMD", "node", "-e", "require('http').get('http://localhost:3003/health',r=>process.exit(r.statusCode===200?0:1)).on('er
      interval: 30s
      timeout: 10s
      retries: 3

  frontend:
    image: 507941515612.dkr.ecr.ap-southeast-2.amazonaws.com/shopsphere-frontend:v1.0.0
    container_name: frontend
    restart: unless-stopped
    networks: [app-network]

  nginx:
    image: nginx:alpine
    container_name: nginx
    restart: unless-stopped
    ports: ["80:80"]
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
    depends_on:
      user-service: {condition: service_healthy}
      product-service: {condition: service_healthy}
      order-service: {condition: service_healthy}
      frontend: {condition: service_started}
    networks: [app-network]

networks:
  app-network:
    driver: bridge

volumes:
  user-data:
  order-data:
```

Compose field	Purpose
image	ECR image URI and version tag
restart	Restart container unless explicitly stopped
environment / env_file	Pass runtime configuration
healthcheck	Check application /health endpoint
volumes	Persist user/order data outside container layer
networks	Private service-to-service communication
depends_on	Wait for backend health before Nginx starts
ports 80:80	Publish Nginx to host TCP/80

Order service calls product-service at http://product-service:3002 using Docker DNS. The user and order services mount named volumes at /app/data. Product data is static in the image in this app version.

12. Environment file and secrets

```
nano .env

NODE_ENV=production
JWT_SECRET=YOUR_GENERATED_64_CHARACTER_SECRET

openssl rand -hex 32
chmod 600 .env
grep -E '^(NODE_ENV|JWT_SECRET)=' .env | \
sed 's/JWT_SECRET=.*/JWT_SECRET=[REDACTED]/'
```

openssl rand -hex 32 generates 32 random bytes encoded as 64 hexadecimal characters. chmod 600 restricts file access to its owner. Keep .env out of Git; commit only a placeholder .env.example. Never paste the actual secret into chat or

documentation. Consider Secrets Manager or Parameter Store for hardened production.

13. Nginx reverse proxy

```
nano nginx/nginx.conf

events {}

http {
    server {
        listen 80;
        server_name _;

        location / {
            proxy_pass http://frontend:80;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }

        location /api/users/ {
            proxy_pass http://user-service:3001;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }

        location /api/products/ {
            proxy_pass http://product-service:3002;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }

        location /api/orders/ {
            proxy_pass http://order-service:3003;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
}
```

Browser path	Nginx upstream
/	frontend:80
/api/users/	user-service:3001
/api/products/	product-service:3002
/api/orders/	order-service:3003

Nginx is the only public entry point. The frontend uses relative API URLs (/api/users/..., /api/products, /api/orders), so the browser talks to the same host and Nginx routes internally. proxy_pass has no trailing slash, preserving the URI path.

14. Validate and deploy

```
cd ~/shopsphere
docker compose config -q
docker compose config --services
docker compose up -d
docker compose ps
```

config -q validates YAML quietly; config --services lists Compose services; up -d creates/starts containers in detached mode. Compose created shopsphere_app-network and named volumes shopsphere_user-data and shopsphere_order-data. The three backend services reported Healthy; frontend and Nginx reported Started/Up.

Frontend and Nginx may show Up rather than Healthy because no healthcheck was defined for them.

15. Application verification

Container status and logs

```
docker compose ps
docker ps
docker compose logs --tail=100
docker compose logs user-service
```

```
docker compose logs product-service
docker compose logs order-service
docker compose logs nginx
```

HTTP tests from EC2

```
curl -i http://localhost/
curl -i http://localhost/api/users/health
curl -i http://localhost/api/products/health
curl -i http://localhost/api/orders/health
curl -i http://localhost/api/products
```

The homepage returned HTTP/1.1 200 OK from nginx with the ShopSphere HTML. The health endpoints and products API were tested. The browser loaded http://13.211.127.214/ and the product catalogue appeared. Registration, login, add-to-cart and order placement were tested successfully.

16. Configure AWS Systems Manager

SSM provides managed remote administration. Session Manager provides browser-based shell access; Run Command executes commands remotely. This lets Jenkins eventually trigger deployment without storing the EC2 SSH private key.

Install SSM Agent

```
sudo snap install amazon-ssm-agent --classic
sudo snap start amazon-ssm-agent
sudo snap services amazon-ssm-agent
```

Initial systemctl checks found neither snap.amazon-ssm-agent.service nor amazon-ssm-agent.service. Snap installation then showed Startup enabled and Current active for amazon-ssm-agent.amazon-ssm-agent.

IAM policy for SSM

Policy	Purpose
ShopSphere-ECR-Pull-Policy	Allow EC2 to authenticate and pull images from ECR
AmazonSSMManagedInstanceCore	Allow the instance to register/connect to Systems Manager

AmazonSSMManagedInstanceCore was attached to the EC2 role while retaining the ECR pull policy. An agent log initially showed credential acquisition failure and AccessDenied involving Default Host Management Configuration. The EC2 Connect screen later showed Ping Online and Session Manager Connected. Ensure the intended instance profile role is attached and carries AmazonSSMManagedInstanceCore; DHMC is an alternative account-level configuration, not automatically required when using an instance profile.

Verify

EC2 → Instances → select ShopSphere-Production-Server → Connect → SSM Session Manager. Observed agent version: 3.3.4793.0; Ping status Online; Session Manager connection status Connected.

17. Session Manager and ssm-user Docker permissions

Session Manager opened a shell as ssm-user. whoami returned ssm-user; hostname returned ip-172-31-37-171. docker ps initially failed with permission denied on /var/run/docker.sock because ssm-user was not in the Docker group.

```
whoami
hostname
docker ps

sudo usermod -aG docker ssm-user
newgrp docker
docker ps
```

After group membership was added and activated, docker ps displayed the five ShopSphere containers. Docker group access is effectively root-level; restrict SSM session access and grant this membership only to trusted administrators.

18. SSM Run Command test

Systems Manager → Run Command → Run command. Select AWS-RunShellScript and target instance i-09225500db5e74575. The commands used:

```
whoami
hostname
cd /home/ubuntu/shopsphere && docker compose ps
```

The invocation returned Overall status Success, Detailed status Success, 1 target, 1 completed, 0 errors and 0 delivery timeouts. Remote command execution was confirmed.

Next planned step, not yet completed: attach a least-privilege SSM deployment policy to the Jenkins IAM identity and add an SSM deployment stage to the existing Jenkins pipeline.

19. Consolidated command reference

Windows PowerShell

```
$key = "$HOME\Downloads\shopsphere-key.pem"
$me = (whoami).Trim()
icacls $key /inheritance:r
icacls $key /grant:r "$($me):(R)"
icacls $key
ssh -i $key ubuntu@13.211.127.214
```

Ubuntu and Docker

```
sudo apt update
sudo apt upgrade -y
sudo systemctl start docker
sudo systemctl enable docker
sudo systemctl status docker
docker --version
docker compose version
sudo docker run hello-world
sudo usermod -aG docker ubuntu
newgrp docker
docker ps
```

AWS CLI, ECR login and pulls

```
aws --version
aws sts get-caller-identity
aws configure get region
aws ecr get-login-password --region ap-southeast-2 | \
docker login --username AWS --password-stdin \
507941515612.dkr.ecr.ap-southeast-2.amazonaws.com
docker pull 507941515612.dkr.ecr.ap-southeast-2.amazonaws.com/shopsphere-user:v1.0.0
docker pull 507941515612.dkr.ecr.ap-southeast-2.amazonaws.com/shopsphere-product:v1.0.0
docker pull 507941515612.dkr.ecr.ap-southeast-2.amazonaws.com/shopsphere-order:v1.0.0
docker pull 507941515612.dkr.ecr.ap-southeast-2.amazonaws.com/shopsphere-frontend:v1.0.0
docker images
```

Deployment and troubleshooting

```
mkdir -p ~/shopsphere/nginx
cd ~/shopsphere
nano docker-compose.yml
nano .env
nano nginx/nginx.conf
chmod 600 .env
docker compose config -q
docker compose config --services
docker compose up -d
docker compose ps
docker ps
docker compose logs --tail=100
docker network ls
docker volume ls
```

API tests and SSM

```
curl -i http://localhost/
curl -i http://localhost/api/users/health
curl -i http://localhost/api/products/health
curl -i http://localhost/api/orders/health
curl -i http://localhost/api/products

sudo snap install amazon-ssm-agent --classic
sudo snap start amazon-ssm-agent
sudo snap services amazon-ssm-agent
sudo snap restart amazon-ssm-agent
sudo snap logs amazon-ssm-agent -n=50
sudo usermod -aG docker ssm-user
newgrp docker
docker ps
```

20. Completion checklist

Task	Recorded status
Create EC2 instance in Sydney	Complete
Configure SSH key permissions	Complete
Connect using SSH	Complete
Install Docker and Compose	Complete
Configure Docker group for ubuntu	Complete
Attach EC2 ECR IAM role	Complete
Authenticate Docker with ECR	Complete
Pull four images	Complete
Create production Compose file	Complete
Create .env	Complete
Create Nginx config	Complete
Deploy five containers	Complete
Verify container health and APIs	Complete
Test registration/login/cart/order	Complete
Install/configure SSM Agent	Complete
Connect using Session Manager	Complete
Test SSM Run Command	Complete
Jenkins IAM permission for SSM	Next / pending
Automated Jenkins deployment	Pending
HTTPS and domain	Pending
Monitoring and alarms	Pending
Rollback strategy	Pending

21. Production-readiness improvements

• **HTTPS:** Add a domain and TLS certificate. The current public URL uses HTTP; do not send real customer credentials over plain HTTP.

• **Database:** User/order data currently uses JSON files in named volumes. This preserves files across container replacement but is not database-grade for concurrent writes, transactions, backups or recovery. Move to PostgreSQL/MySQL or another suitable database before real production.

• **Secrets:** Use Secrets Manager or Parameter Store with least-privilege retrieval.

• **Monitoring:** Configure CloudWatch logs, metrics, alarms and notifications.

• **Backups:** Implement and test backup/restore for persistent data.

• **Security:** Review IAM least privilege, SSH exposure, security groups, Docker group access, patching and image vulnerabilities.

• **Deployment reliability:** Add post-deployment health verification and rollback to Jenkins.

• **Availability:** One EC2 instance is a single point of failure. Consider load balancing and multiple instances/AZs if requirements and budget justify them.

Cost reminder: EC2 compute, EBS, ECR storage, data transfer and some Systems Manager capabilities can incur charges. Check current Free Plan credits, service pricing and billing. Stopping EC2 does not necessarily stop EBS or other charges.

22. Interview questions

• Why did you choose EC2 instead of Elastic Beanstalk?

• What is the difference between an IAM user and an IAM role?

• Why attach an IAM role to EC2 instead of storing access keys?

• How does Docker authenticate with Amazon ECR?

• What is the difference between a Docker image and a container?

• Why use Docker Compose?

• What is a Docker bridge network?

• Why use Nginx as a reverse proxy?

• What is the difference between a Docker volume and a bind mount?

• What is a Docker healthcheck?

• How does SSM Session Manager work without SSH?

• What is the difference between Session Manager and Run Command?

• Why should backend ports not be publicly exposed?

• How would you automate this deployment through Jenkins?

Technology	Role in this project
EC2	Compute host
IAM role/instance profile	Temporary AWS permissions for EC2
ECR	Private image registry
Docker	Container runtime
Docker Compose	Multi-container orchestration on one host
Nginx	Public web server and reverse proxy
Bridge network	Private connectivity between services
Named volumes	Persistence for user/order JSON files
SSM Session Manager	Browser-based managed shell
SSM Run Command	Remote command execution
Jenkins	CI pipeline; SSM-based CD is next

Final summary and accuracy notes

ShopSphere was deployed to Ubuntu 24.04 on EC2 in ap-southeast-2. Jenkins had published four images to ECR. EC2 authenticated using its IAM instance role, pulled the images and Docker Compose started the frontend, three backend services and Nginx. Nginx exposed port 80 and routed frontend/API traffic to the correct containers. Browser and API tests, including registration, login, cart and order flow, succeeded. SSM Agent was installed; Session Manager connected; Docker access was granted to ssm-user; and an AWS-RunShellScript Run Command succeeded.

Next planned milestone: Configure Jenkins IAM permissions for SSM and automate deployment after a successful build and ECR push.

Accuracy note: Deployment values and observed outcomes are based on the conversation. Compose and Nginx examples are reconstructed reference configurations, not verified byte-for-byte copies of the live files. Docker installation commands are a reproducible method, not a claim that each exact command was typed in the original session. Validate files and permissions before reuse.